Ambrose Tuscano

|ambrosetuscanojobs@gmail.com|(443)-627-1151|LinkedIn.com/ambrosetuscano/|

TECHNICAL SKILLS

- **Core Expertise:** Distributed systems design, high-throughput data pipelines, serverless architecture, API development and modeling, ML/AI integration, infrastructure and performance optimization, real-time event-driven systems, and observability and monitoring.
- **Languages:** Java, Python, SQL, TypeScript, JavaScript.
- **Tools:** Asana, Docker, Git, Grafana, Postman.
- **Databases & Storage:** Apache Iceberg, DynamoDB, Memcached, MongoDB, MySQL, PostgreSQL, RDS, Redshift, Redis, S3.
- **Frameworks & Libraries:** Dagger, Spring Boot, Guice, GraphQL, Hibernate, Django, FastAPI, Flask, ReactJS, TensorFlow, Vert.x.
- **AWS Services:** Apache Kafka (Amazon MSK), API Gateway, Athena, Bedrock, CloudFormation, CloudWatch, EC2, ECS, ElastiCache, Fargate, Glue, Kinesis Firehose, Lambda, QuickSight, SageMaker, SNS, SQS, Step Functions.

PROFESSIONAL EXPERIENCE

Software Engineer 2 @ Microsoft — **Redmond, WA | Apr 2026 - ongoing**

- Designed, tested, and rolled out new graph-relationship rules for Microsoft's cloud resource inventory graph — used by security, governance, and compliance systems across Azure — taking each rule end-to-end from implementation through region-by-region production rollout; the primary rule matched ~128K–150K resource snapshots per day (across ~30 customer tenants) after promotion.
- Improved observability of a high-throughput distributed relationship-processing pipeline: added per-record dimensions to the primary throughput metric, fixed metric-name collisions and label mis-tagging affecting 7 telemetry signals across three microservices, and built new operational dashboards for the deletion-anomaly monitors.
- Contributed to a five-region compute scale-out (East US, Southeast Asia, West Europe, North Europe, West US 2) of the graph-generation service that ~doubled processing capacity with no additional cloud spend, and drove the buildout of the same distributed infrastructure stack in a net-new Azure region (Australia East).

Sr. Generative AI Engineer @ Nissan Motor Corporation — **Franklin, TN | Feb 2026 - Apr 2026**

- Designed and shipped Tell Me More, Nissan's first AI product for the dealership business: a production multi-agent chatbot integrated into the dealership appointment pipeline. Architected a four-agent system (Conversation, Rubric, Vehicle History, Dealership Internal) on AWS Bedrock AgentCore with persistent session memory, supporting multi-concern diagnostic intake flows end-to-end.
- Owned end-to-end business engagement: gathered requirements directly from dealership stakeholders, iterated on product behavior based on feedback, and drove alignment across business and technical teams.
- Owned the full AWS infrastructure, chat application backend, and partner integrations, onboarding X dealership channels.

Software Engineer @ Amazon [Advertising] — **Arlington, VA | May 2022 - Jan 2026**

- Designed and built a scalable Ad Labeling Service to process and store rule and AI-driven labels, increasing throughput by 5× over the legacy system and enabling extensible support for sync and async labeling services to meet post-bid filtering needs for publishers.
- Engineered a serverless data lake using DynamoDB PITR exports to Apache Iceberg tables, cataloged in AWS Glue and queryable via Athena, achieving 30-minute data freshness SLA for analytics workloads.
- Built an agent-based validation framework using model-driven control logic (MCP) to evaluate ad labels against centralized label metadata, schemas, and semantic constraints, combining rule-based inference, automated decisioning, and metadata lookups to prevent invalid or inconsistent labels from propagating into downstream ad serving systems.
- Built a Slack-integrated AWS Bedrock chatbot using a RAG-based architecture, leveraging Amazon Kendra for document retrieval and Bedrock for grounded responses, enabling teams to track tickets and metrics, accelerate onboarding, and provide leadership with real-time visibility into system and client status.
- Built Step Functions workflows for real-time ad evaluation (5K+ TPS, sub-second latency), enforcing pre-bid filtering rules that validate ad labels against publisher requirements for compliance and yield optimization.
- Contributed to coding and infrastructure development for Ad Registration Service APIs, data models, and GraphQL layers, alongside a scalable ElastiCache-based ingestion system. Achieved low-latency APIs (<15 ms) supporting 1,000+ TPS, enabling efficient access to advertiser and creative metadata and addressing key publisher concerns in ad bidding.
- Supported the launch, end-to-end testing, and Weblab based A|B test experimentation of the Ad Relevance API and Infrastructure for Amazon Retail, achieving over 1 million (TPS) and a projected year-over-year $10 million increase in ad revenue for Amazon DSP.
- Wrote a root cause analysis, and resolved a 10% request drop issue by optimizing infrastructure scaling policies, also enhanced alarming and monitoring, and implemented CI/CD best practices to help test and catch issues earlier.
- Optimized the BrandSafety data pipeline throughput by 3x (from 17 million to 52 million products processed per day). This improvement drastically reduced a critical backfill process from 45 days to under 14 days.
- Orchestrated the successful transition of a complex service with six interdependent subsystems to a new team, leading comprehensive documentation, knowledge transfer sessions, and post-handoff support while maintaining 100% service uptime.

Software Engineer @ Openprise **Arlington, TX | Feb 2022 - Mar 2022**

- Developed Python-based API calls for seamless integration with the Google Calendar API, enabling efficient data consumption.
- Implemented data pre-processing techniques and successfully ingested the processed data into a PostgreSQL database, ensuring optimal data organization and accessibility.

ML Engineering Intern @ NXP Semiconductors **San Jose, CA | May 2021 - Aug 2021**

- Revamped the test suite using Python and Keras, incorporating advanced Machine Learning techniques in NLP and Unsupervised learning. Achieved a 30% reduction in runtime and decreased manual intervention by 60%.
- Collaborated with multiple teams to assess software testing needs, identified opportunities for ML integration. Explored SaaS solutions and authored comprehensive documentation to support seamless adoption within the team.

Graduate Research Assisstant @ University of Maryland, Baltimore County **Baltimore, MD | Aug 2020 - Dec 2021**

- Researched and fine-tuned transformer-based NLP models (BERT) for fake vs. real news classification on imbalanced multimodal datasets, building scalable preprocessing pipelines for text tokenization, image feature extraction, and video frame analysis, performing multimodal feature fusion, and evaluating model performance using precision, recall, F1 score, and confusion matrices.

EDUCATION

Master of Science (M.S.) in Computer Science | University of Maryland, Baltimore County **Jan 2020 - Dec 2021**

Bachelor of Engineering(B.E.) in Computer Science | University of Mumbai **May 2015 - May 2019**

KEY PROJECTS

Tell Me More — Agentic Car Diagnostic Chatbot:

[AWS Bedrock AgentCore, Strands, Multi-Agent Systems, FastAPI, RDS, Python]

- Built Nissan's first AI product for the dealership business: a production four-agent system that triggers on dealership appointment booking, sending customers a personalized link to complete a structured diagnostic intake conversation before their visit.
- The Conversation Agent manages customer dialogue, a priority-ordered concern queue, and orchestration of three specialist agents as tools. The Rubric Agent handles real-time field extraction and answer quality assessment. The Vehicle History Agent surfaces relevant past repair context. The Dealership Agent checks warranty coverage, parts availability, and dealer-specific advisories.
- Each customer concern runs its own full intake cycle with persistent agent memory across turns. Integrated with internal Nissan systems and five dealership partner channels.

Multimodal AI System for Ad Similarity Detection:

[AWS AI (Transcribe, Rekognition, Bedrock), LangChain, OpenSearch]

- Built multimodal AI pipeline that extracts and analyzes content from ads (audio, video, images, text) using AWS Transcribe for audio-to-text, Rekognition for visual descriptions, and Bedrock to generate unified textual representations for similarity comparison.
- Developed agentic workflow using LangChain to orchestrate AWS AI services for processing audio, video frames, analyzing images, and combining results into comprehensive text descriptions that are converted to vector embeddings and stored in OpenSearch.
- Implemented dual similarity detection capabilities: (1) k-nearest neighbor search in OpenSearch to find all similar ads for a given ad, and (2) pairwise comparison of specific ad IDs by computing cosine similarity scores to determine duplicates.

Enterprise GenAI Knowledge & Search Platform

[Python, AWS (Bedrock, Kendra, Lambda), MCP, FastMCP, LangChain, LangGraph, AutoGen, SQL, REST APIs]

- Built a production-grade RAG-based GenAI knowledge platform using Python and Amazon Kendra to retrieve enterprise documents, and AWS Bedrock to generate accurate, context-aware responses for organization-wide natural language search.
- Designed and **implemented MCP servers using FastMCP**, exposing enterprise data sources such as **Kendra (documents), CloudWatch (metrics), Asana (task tracking), ticketing systems, and internal services** as standardized **MCP tools and resources** for agent-driven consumption.
- Developed a **multi-agent orchestration layer** using **LangChain, LangGraph, and AutoGen**, enabling agents to interpret user intent, dynamically select MCP tools, invoke FastMCP endpoints, aggregate multi-source results, and synthesize final responses using **LLM reasoning via Bedrock**.
- Implemented agentic decision-making logic where agents determine which data sources to query based on user intent, aggregate information from multiple systems, and synthesize comprehensive answers using Bedrock's LLM capabilities.

Peer to Peer Distributed File System:

[Python, Sockets, Multi-threading]

- Architected a Distributed System using TCP protocol, enabling seamless File CRUD operations across multiple P2P connected nodes.
- Implemented a robust failure resilient architecture by incorporating replication strategy and employing version control techniques.